

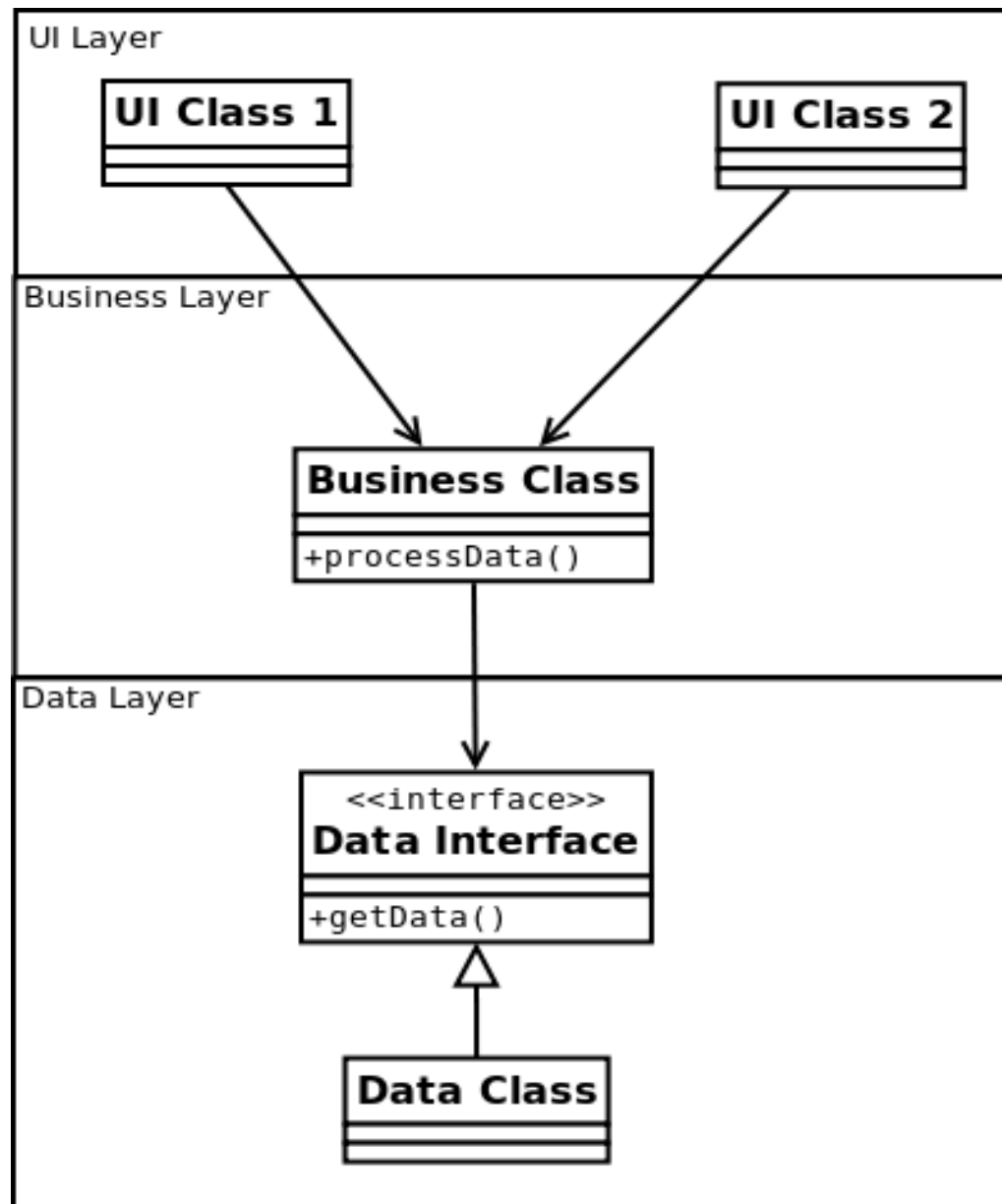
Layered Architecture

Layered Architecture

- Application is divided into logical layers
- Each layer groups components of similar nature/role (e.g. UI, Business, Data, etc.)
- Each layer builds on top of lower layers and communicate with its adjacent neighbors through well-defined interfaces
- Allows possibility of substituting one layer with another equivalent one

Data Access Object

- A design pattern to manage data storage in general
- Provides an abstract interface to possibly multiple and diverse storage systems
- Use general data-structures
 - Need not be dependent on application logic
- Critical component of 3-Layered Architecture



Example 1: Contacts Application (Data Access Layer)

```
public interface IDAO {  
  
    public boolean save(Hashtable<String,String> attributes);  
    public ArrayList<Hashtable<String,String>> load();  
    // other functions ...  
}
```

```
public class ContactDbDAO implements IContactDAO {  
  
    public boolean save(Hashtable<String,String> attributes)  
    {  
        // write to database  
    }  
  
    public ArrayList<Hashtable<String,String>> load()  
    {  
        // load from database  
    }  
  
}
```

```
public class ContactFileDAO implements IContactDAO {  
  
    public boolean save(Hashtable<String,String> attributes)  
    {  
        // write to file  
    }  
  
    public ArrayList<Hashtable<String,String>> load()  
    {  
        // load from file  
    }  
  
}
```

```

public class Contact {

    // other attributes ...
    IDAO dao = new ContactDbDAO();

    public boolean save(){
        Hashtable<String,String> data = new Hashtable<>();
        data.put("id",id);
        data.put("firstname",firstName);
        // other attributes
        return dao.save(data);
    }

    public static ArrayList<Contact> getContacts(){

        ArrayList<Contact> contacts = new ArrayList<> ();
        ArrayList<Hashtable<String,String>> data = dao.load();

        for(Hashtable<String,String> d : data){
            contacts.add(new Contact(d.get("id"),d.get("firstname"),
                                     // other attributes
                                     ));
        }

        return contacts;
    }

}

```

Business Layer

Example 2: Graphics Application

```
public interface IShapeDAO {  
  
    public void insertShape(String type,Point point,Hashtable<String,String> shapeAttributes);  
    public ArrayList<Hashtable<String,String>> getShapes();  
  
}
```

```
public class ShapeDAO implements IShapeDAO {  
  
    public void insertShape(String type,Point point,Hashtable<String,String> shapeAttributes)  
    {  
        // write to storage (database / file)  
    }  
  
    public ArrayList<Hashtable<String,String>> getShapes()  
    {  
        // load from storage (database / file)  
    }  
  
}
```

Data Access Layer

```
public abstract class Shape {  
  
    ...  
    ...  
  
    public abstract void save();  
  
    public static ArrayList<Shape> loadShapes(){  
  
        ArrayList<Shape> shapesCollection = new ArrayList<Shape> ();  
  
        IShapeDAO dao = new ShapeDAO();  
        ArrayList<Hashtable<String,String>> shapes = dao.getShapes();  
  
        for(Hashtable<String,String> shape : shapes){  
            shapesCollection.add(ShapeFactory.getInstance().createShape(shape));  
        }  
  
        return shapesCollection;  
    }  
}
```

Business Layer

```
public class Rectangle extends Shape {  
  
    ...  
    ...  
  
    public void save(){  
        IShapeDAO dao = new ShapeDAO();  
  
        Hashtable<String,String> attributes = new Hashtable<String, String>();  
        attributes.put("width", String.valueOf(width));  
        attributes.put("length", String.valueOf(length));  
  
        dao.insertShape("rectangle", point, attributes);  
    }  
}
```

Object Relational Mapping

- Possibilities
 - One table per class hierarchy
 - One table per concrete class
 - One table per class
 - Generic table structure

Design Databases according to Data Design techniques.
Objects can be mapped to Databases